3. 그래프 (1) CSED331 · 알고리즘 전체 홈↗

이 페이지

1. 그래프 기초
2. 그래프 표현 — 인접 행렬 vs 인접 리스트
3. 경로, 사이클, 연결성
4. 트리
5. DFS
6. BFS
7. DFS vs BFS
8. 방향 그래프의 DFS — pre/post와 간선 4종류
9. cycle과 DAG
10. 연습문제

그래프 (1) — 기초, 표현, DFS/BFS

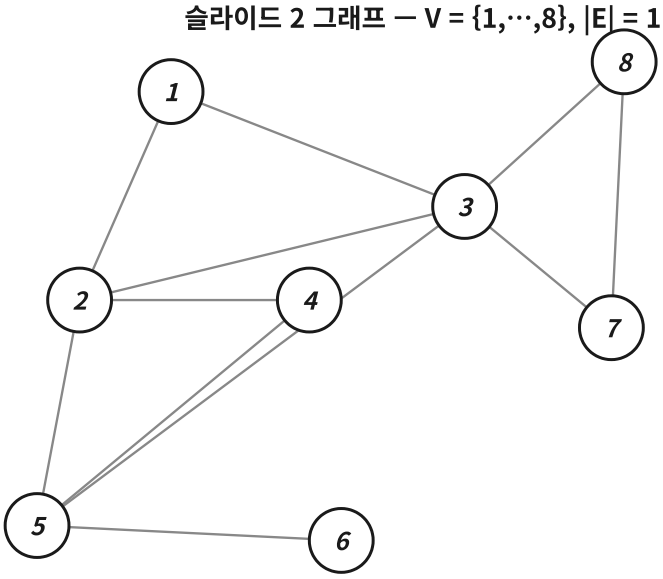
2026-09-16 (수) · 09-21 (월) 앞부분 · 슬라이드 Graph Algorithms (p.1–18, 녹취 없음, 교수 marked PDF)

그래프 기초

graph $G = (V, E)$ — 노드(vertex) 집합 V + 간선(edge) 집합 E. 간선은 서로 다른 두 노드를 잇는 쌍 (자기 자신끼리는 X).

- 좌표·굽은 모양은 무관 — 노드와 연결이 같으면 **같은 그래프** (위상만 봄)
- 간선 길이(거리)는 일단 안 봄 → 뒤에서 가중치를 붙임

슬라이드 2 그래프 — V = {1,⋯,8}, |E| = 11 (오른쪽: 인접 리스트)



1 (deg 2) : 2 → 3

2 (deg 4) : 1 → 3 → 4 → 5

3 (deg 5) : 1 → 2 → 5 → 7 → 8

4 (deg 2) : 2 → 5

5 (deg 4) : 2 → 3 → 4 → 6

6 (deg 1) : 5

7 (deg 2) : 3 → 8

8 (deg 2) : 3 → 7

차수의 합 = 22 = 2 × 11

- **응용** — 관계를 노드와 간선으로 바꾸면 알고리즘을 그대로 씀: 웹(페이지/하이퍼링크), SNS(계정/팔로우), 회로(게이트/전선), 내비게이션(교차로/도로 → 최단 경로)
- **PageRank** — 링크 1개 = 지지표 1개. 순위는 링크를 건 페이지들의 PageRank에 따라 재귀적으로 결정. 많이 받을수록, 높은 페이지에서 받을수록 순위 ↑. 링크를 따라 이동하는 무작위 서퍼(random surfer) 모델에서 그 페이지를 장기적으로 방문할 확률로 해석한다. 표준 모델은 일정 확률로 링크와 무관하게 임의의 페이지로 이동하는 damping(teleportation)을 포함한다. 단, 이는 개념적 직관이며 구체적인 계산에는 별도의 수식과 가정이 필요하다

교차 그래프 — reduction의 전형

- 선분(원)마다 노드 1개, **서로 교차하면 간선** — 교차 문제가 그래프 문제로 바뀜
- 흐름: 원래 문제 → 그래프 문제 → 그래프 알고리즘 → 해를 원래 문제로 되돌림 (재귀 장의 reduction과 같은 패턴)

그래프 표현 — 인접 행렬 vs 인접 리스트

항목	인접 행렬	인접 리스트
구조	$n \times n$ 배열, 간선이면 $a_{ij} = 1$	노드마다 이웃 linked list

간선 하나	두 번 저장 (a_{ij}, a_{ji})	두 번 저장 (양 끝점의 리스트)
공간	$O(n^2)$ (대칭이라 절반은 낭비)	$O(n+m)$
$(u,v) \in E$ 확인	$O(1)$	$O(\min(\deg u, \deg v))$ — 둘 중 짧은 리스트를 훑음 — (5, 7) 확인 시 5의 리스트(4칸)보다 7의 리스트(2칸)
모든 간선 찾기	$\Theta(n^2)$	$\Theta(n+m)$
잘 맞는 그래프	dense	sparse

- 이 표는 무방향 그래프 기준이다. 방향 그래프에서는 간선을 출발점의 리스트에만 저장하며, (u, v) 확인은 u의 나가는 간선 리스트를 훑어 O(deg⁺(u))가 걸린다.
- $n = |V|, m = |E|$. 리스트 길이의 합 = 차수의 합 = $2m$ (간선마다 양 끝점 리스트에 한 번씩)
- 리스트 크기는 간선 수에 민감 — 간선이 적으면 n 수준, 많으면 n^2
- 슬라이드 2 그래프: 차수 합 2+4+5+2+4+1+2+2 = 22 = 2×11. 행렬의 1의 개수도 22
- trade-off — 공간을 아끼면 시간이 늘어남. 리스트는 인덱스로 바로 못 찾아 앞에서부터 훑음
- SNS 규모: 사용자 수십억 × 수십억 칸 행렬은 저장 불가. 친구는 수백 명뿐 → 거의 전부 0 → **리스트가 맞음**

경로, 사이클, 연결성

- **path** — 연속한 두 노드가 모두 간선으로 이어진 노드 열 $v_1 v_2 \cdots v_k$
- **simple path** — 노드가 모두 서로 다른 path
- **cycle (무방향 단순 그래프)** — 시작 노드와 끝 노드가 같은 닫힌 경로이다. 길이(간선 수)는 3 이상이어야 하며, 끝 노드를 제외한 나머지 노드들은 모두 서로 다르다. 즉, 시작 노드로 돌아오되 그 사이에 노드를 반복해서 방문하지 않는 경로이다.
- **cycle (방향 그래프)** — 모든 간선을 화살표 방향으로 따라 시작 노드로 돌아오는 닫힌 경로이다. 시작 노드와 끝 노드는 같고, 끝 노드를 제외한 나머지 노드들은 모두 서로 달라야 한다. 자기 자신을 잇는 간선이 없는 경우 길이 2의 cycle도 가능하다. 예를 들어, (u→v→u)는 (u→v)와 (v→u)가 모두 존재할 때 방향 그래프의 cycle이다.
- **connected** — 모든 노드 쌍 사이에 path가 있음. 방향 그래프에서는 모든 순서쌍에 대해 양방향 path가 있으면 strongly connected, 방향을 무시한 무방향 그래프가 connected이면 weakly connected라고 한다

- 슬라이드 2 그래프: 5-2-3-7 (simple path O) · 5-1 (간선 없음 → path X) · 5-4-2-5-6 (path O, simple X) · 5-4-2-5 (cycle) · 3-7-8-3 (cycle)
- 슬라이드 8 그래프: 노드 11개, 연결요소 2개 ({1,⋯,8}, {9,10,11}). 5→8은 path 있음, 5→10은 없음 → connected X. 그래프가 크면 눈으로 판단 불가 → **탐색 알고리즘이 필요**

트리

노드 n개인 무방향 그래프에서 아래 셋 중 **둘이 성립하면 나머지 하나도 성립**:

(a) connected (b) cycle 없음 (c) 간선이 $n-1$개

- **tree** = connected + acyclic인 무방향 그래프 ⇔ 모든 노드 쌍 사이에 **simple path가 정확히 하나**
- 간선 수로 판정할 때 (유한 무방향 그래프): connected이면서 간선이 n−1개이면 tree이다. 간선이 n−1개보다 적으면 connected일 수 없고, n−1개보다 많으면 connected 여부와 무관하게 cycle이 존재한다. 간선이 n−1개라는 것만으로는 부족하고, connected인지까지 확인해야 tree라고 할 수 있다

- **방향 그래프는 간선 수로 판단 불가** → 뒤의 DFS back edge 유무로 사이클 존재 여부를 판단. 단, back edge가 없다는 사실만으로 방향 그래프가 트리라고 결론 내릴 수는 없다 — 방향 트리의 판정에는 별도의 구조적 조건이 필요하다

unique path 증명 보기

- tree에서 두 노드 사이 path가 둘이면 두 path의 합집합에 cycle이 생김 → acyclic과 모순
- 반대로 모든 쌍 사이에 path가 있으면 connected, path가 유일하면 cycle 불가 → acyclic
- 간선 수 $n-1$은 노드 수에 대한 귀납법 (base: 노드 1개, 간선 0개)
- 슬라이드 9 그래프: $n=11$, 간선 10개, connected, cycle 없음 → tree

DFS

질문 — 시작 노드 s에서 도달 가능한 부분은 어디인가. **아이디어** — 안 가 본 방향으로 끝까지 파고, 막히면 직전 노드로 돌아와 계속.

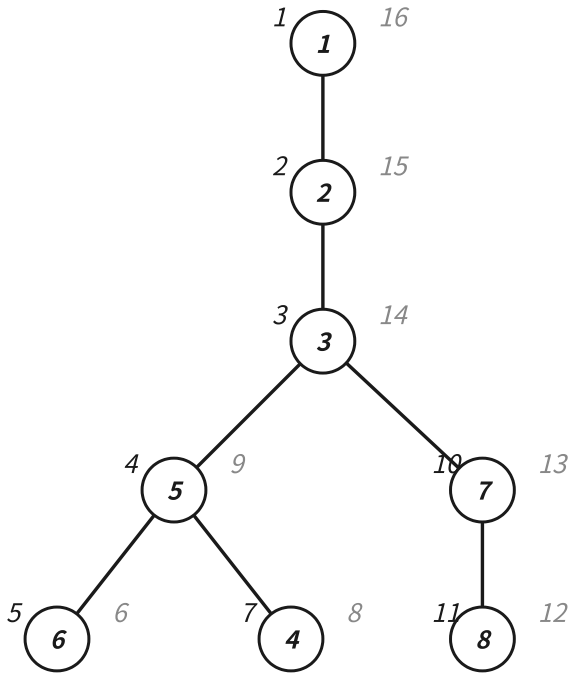
```
explore(G, v):
    visited(v) = true
    이웃 u마다: visited(u) = false이면 explore(G, u)

dfs(G):
    모든 노드 visited = false
    방문 안 한 노드마다 explore
```

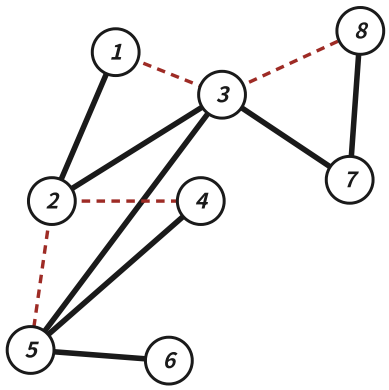
- 재귀 호출 = **stack** (호출 스택이 돌아갈 곳을 기억). visited는 배열, 처음엔 전부 false
- explore 한 번 = **연결요소 하나**만 훑음 (무방향 그래프에서 미방문 정점으로 시작할 때). dfs(G)가 다른 연결요소에서 다시 시작 (슬라이드 8 그래프는 루트 1, 9에서 2번). 방향 그래프에서는 한 번의 explore가 그 정점에서 방향 경로로 도달 가능한 정점들을 방문한다
- 시간 계산은 관점을 노드에서 간선으로 바꿔 세면 쉬움 — 간선마다 끝점이 2개라 총 $2|E|$번 검사

손계산: 슬라이드 그래프에서 DFS (시작 1)

DFS (시작 1) — 방문 1, 2, 3, 5, 6, 4, 7, 8 · 옆 숫자: pre / post



non-tree 간선 4개 (전부 조상-자손)



굵은 선 = tree edge 7개 (= n−1)
빨강 점선 = non-tree · 무방향 DFS엔 cross 없음

t	사건	사건 직후 호출 스택 (바닥 → 위)
1	explore(1) 시작	1
2	explore(2) 시작	1 2
3	explore(3) 시작	1 2 3
4	explore(5) 시작	1 2 3 5
5	explore(6) 시작	1 2 3 5 6
6	6 끝 (이웃 5는 방문됨)	1 2 3 5
7	explore(4) 시작	1 2 3 5 4
8	4 끝	1 2 3 5
9	5 끝	1 2 3
10	explore(7) 시작	1 2 3 7

11	explore(8) 시작	1 2 3 7 8
12	8 끝	1 2 3 7
13	7 끝	1 2 3
14	3 끝	1 2
15	2 끝	1
16	1 끝	(비어 있음)

노드	**1**	**2**	**3**	**5**	**6**	**4**	**7**	**8**
pre (시작 t)	1	2	3	4	5	7	10	11
post (끝 t)	16	15	14	9	6	8	13	12
parent	–	1	2	3	5	5	3	7

- 5에서는 그림 기준 6을 먼저 방문. 리스트 순서(2, 3, 4, 6)대로면 1, 2, 3, 5, 4, 6, 7, 8 → **DFS 트리는 유일하지 않음**
- tree edge 7개 $= n - 1$: (1,2) (2,3) (3,5) (5,6) (5,4) (3,7) (7,8). non-tree 4개: (1,3) (2,4) (2,5) (3,8) — 전부 조상-자손 관계 (**무방향 DFS에는 cross 간선이 없음**)
- 간선 검사 횟수 = 차수의 합 = 22 = $2|E|$

정확성과 시간

- 이웃으로만 움직임 → 도달 못 하는 곳으로 점프하지 않음. 도달 가능한 노드는 하나도 놓치지 않음 (아래 증명)
- 각 간선을 **정확히 2번** 검사: explore(u)에서 한 번, explore(v)에서 한 번. 노드마다 반복 수 = 차수 → 합 = $2m$
- 이 셈은 **인접 리스트** 기준이다: 무방향 그래프에서는 각 간선이 양 끝점의 리스트에 저장돼 인접 항목이 총 2|E|개이고, 방향 그래프에서는 간선이 한 번만 저장돼 |E|개이다. 어느 쪽이든 DFS·BFS의 시간은 Θ(|V|+|E|). **인접 행렬**로 표현하면 이웃을 찾을 때 행 전체를 훑어야 하므로 Θ($|V|^2$)이 된다
- 노드 방문 $|V|$ + 간선 검사 $2|E|$ →

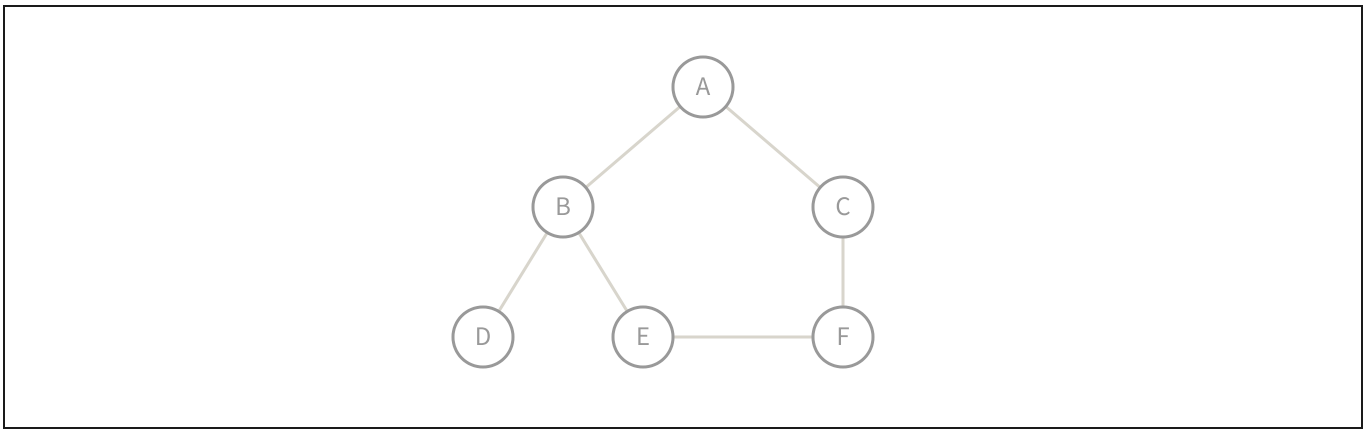
  $$O(|V| + |E|)$$

  인접 리스트로 주어진 **전체 그래프를 순회**하는 문제에서는 입력 전체를 읽어야 하므로 이 시간이 최적(optimal)이다. 단, 단일 시작 정점의 도달 영역만 찾는 문제의 최적성은 입력 모델과 도달 영역에 따라 별도 논증이 필요하다

도달 가능한 노드는 반드시 방문함 (증명)

- u가 v에서 도달 가능한데 explore가 u를 놓쳤다고 가정
- v에서 u까지 아무 path π를 잡고, π 위에서 방문된 노드 중 마지막을 z, 그 다음을 w라 함
- z를 방문할 때 모든 이웃을 확인하므로 w를 발견하고 이동했을 것 → w도 방문됨 → z가 마지막이라는 가정과 모순

깊이 우선 탐색 (DFS)



A에서 시작. DFS는 스택(LIFO)으로 **깊은 곳부터** 방문.

◀ 이전 1 / 8 다음 ▶

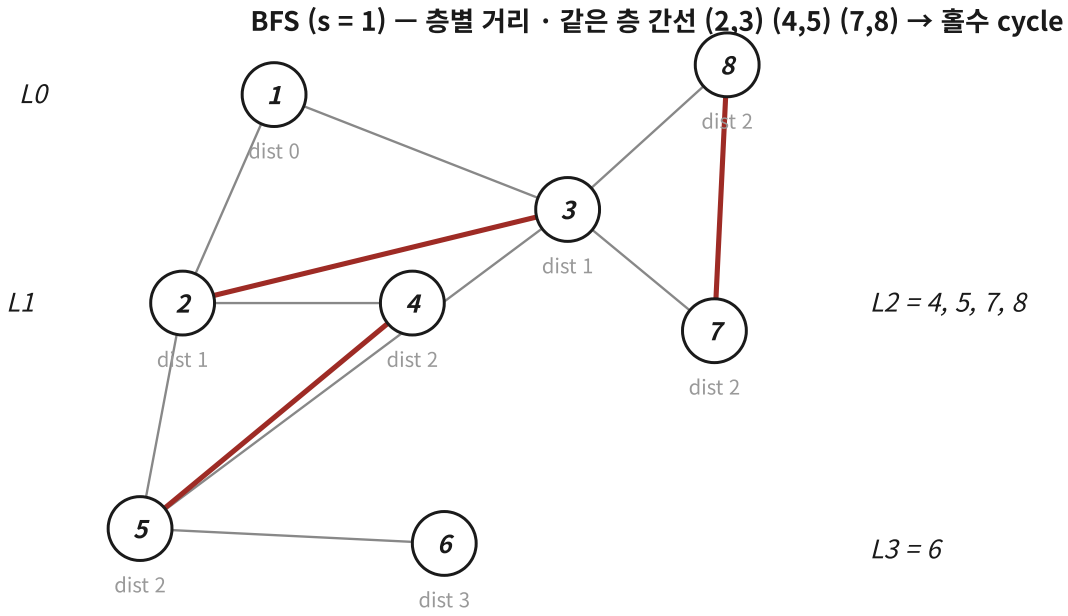
BFS

s에서 **가까운 순서대로 층별로** 탐색하며 거리 계산. layer 0 = 나, layer 1 = 직접 친구, layer 2 = 친구의 친구, ⋯ DFS처럼 끝까지 파지 않고 모든 방향으로 한 단계씩.

층의 정의

$$L_0 = \{s\}, \quad L_{i+1} = \{v \notin L_0 \cup \cdots \cup L_i : v\text{는 } L_i\text{의 어떤 노드와 인접}\}$$

- 이하 두 관찰은 **무방향 그래프**에서 BFS를 수행할 때 성립한다
- **관찰 1** — 모든 간선은 같은 층 또는 연속한 두 층의 노드를 이음
- **관찰 2** — 같은 층의 두 노드를 잇는 간선이 있으면 **홀수 길이 cycle**이 있음
- 슬라이드 그래프의 같은 층 간선 (2,3), (4,5), (7,8) → 삼각형 1-2-3, 2-4-5, 3-7-8 (길이 3 = 홀수 cycle)



방문 순서: 1, 2, 3, 4, 5, 7, 8, 6 · 간선은 같은 층 또는 연속한 두 층만 이음

같은 층 간선 ⇒ 홀수 cycle (증명)

- 무방향 그래프에서, u와 v가 같은 층 k에 있고 간선 (u, v)가 있다고 하자. 두 노드의 BFS 트리 경로를 거슬러 올라가면 가장 가까운 공통 조상 w가 있고, w는 어떤 층 j에 있다
- w에서 u로, w에서 v로 내려오는 트리 경로는 각각 층을 하나씩 내려오므로 길이가 k−j이다
- 두 경로에 간선 (u, v)를 더하면 길이가 2(k−j)+1인 cycle이 되므로, 홀수 길이 cycle이 존재한다

---

bfs 알고리즘과 큐 추적

```
bfs(G, s):
    모든 u: dist(u) = ∞;  dist(s) = 0;  Q = [s]
    Q가 비지 않는 동안:
        u = eject(Q)
        u의 이웃 v마다: dist(v) = ∞이면
            inject(Q, v);  dist(v) = dist(u) + 1
```

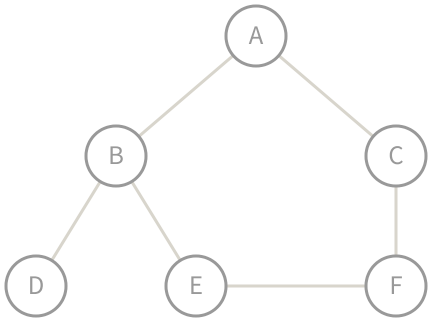
- dist가 ∞가 아니면 이미 큐에 들어간 노드 → **노드는 큐에 딱 한 번**. dist 값이 방문 표시 역할
- 큐는 FIFO → 층 순서대로 나옴. 큐 안엔 거리 d, $d+1$인 노드만 섞여 있음
- 시간: 초기화 $O(|V|)$ + 이웃 훑기 $2|E|$ → $O(|V|+|E|)$ (DFS와 같음). **간선 길이가 모두 1이면 dist = 최단 거리**

단계	eject	inject	큐
0	–	–	[1]
1	1	2, 3	[2 3]
2	2	4, 5	[3 4 5]
3	3	7, 8	[4 5 7 8]
4	4	–	[5 7 8]
5	5	6	[7 8 6]
6	7	–	[8 6]

7	8	–	[6]
8	6	–	[ ]

- 방문 순서 1, 2, 3, 4, 5, 7, 8, 6. dist: 1→0 / 2, 3→1 / 4, 5, 7, 8→2 / 6→3

너비 우선 탐색 (BFS)



A에서 시작. BFS는 큐(FIFO)로 **가까운 정점부터** 방문.

◀ 이전 1 / 8 다음 ▶

DFS vs BFS

항목	DFS	BFS
자료구조	stack (재귀 호출)	queue
탐색 모양	깊게 파고듦	넓고 얕게, 거리가 늘어나는 순서
다른 연결요소	한 시작점에서는 도달 가능한 노드만 방문. 전체 탐색에서는 미방문 노드에서 다시 시작해 모든 연결요소를 방문	한 시작점에서는 도달 가능한 노드만 방문. 전체 탐색에서는 미방문 노드에서 다시 시작해 모든 연결요소를 방문
거리	최단 거리 보장 안 함 (DFS 트리 ≠ 최단 거리)	간선 길이 1이면 dist = 최단 거리
시간	$O(n+m)$	$O(n+m)$

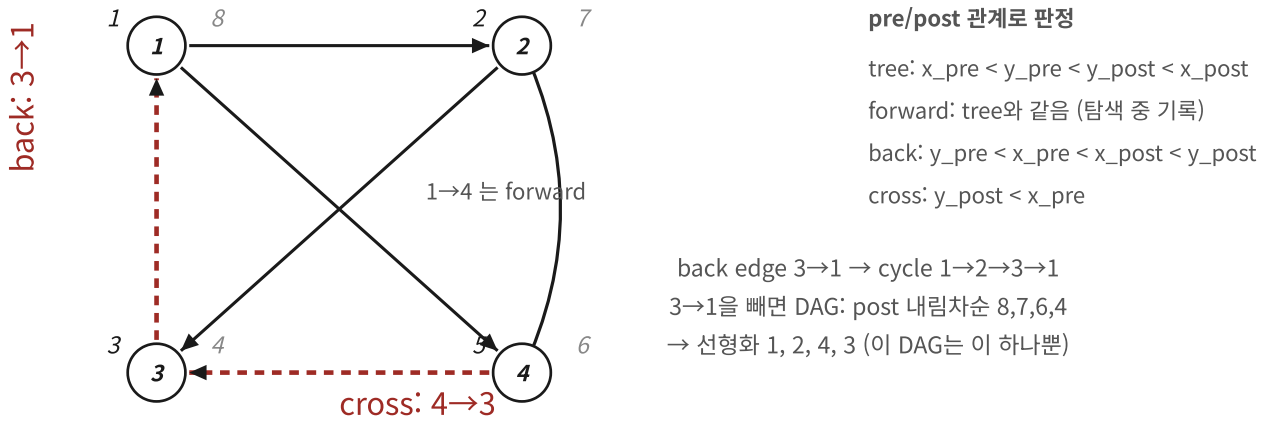
비유	미로에서 막힐 때까지 한 길로 가다 되돌아옴	내 주변부터 한 칸씩 넓혀 가며 확인

- 같은 그래프, $s = 1$: DFS 1, 2, 3, 5, 6, 4, 7, 8 / BFS 1, 2, 3, 4, 5, 7, 8, 6
- 시간은 같지만 쓰임이 다름 — 도달 가능성·연결요소·cycle 검사 → **DFS**, 가까운 순서·최단 거리 → **BFS**

방향 그래프의 DFS — pre/post와 간선 4종류

- 방향 그래프에서도 DFS 그대로 — 간선을 **화살표 방향으로만** 따라감. DFS 트리 용어: root, ancestor, descendant, parent, child
- **pre / post** — explore 시작·끝 때의 시계 값 (시계는 시작·끝날 때 각각 1씩 증가)

방향 그래프 DFS (시작 1) — pre/post와 간선 4종류



종류	뜻	판정 시점	pre/post 관계	예
tree	DFS가 따라가 발견한 간선	y 미방문	$x_{pre} < y_{pre} < y_{post} < x_{post}$	1→2, 2→3, 2→4
forward	조상 → 자손, tree 아님	y는 이미 끝났고 x보다 늦게 시작	tree와 같음	1→4
back	자손 → 조상	y가 아직 진행 중 (스택에 있음)	$y_{pre} < x_{pre} < x_{post} < y_{post}$	3→1
cross	서로 조상·자손 아님	y가 이미 끝났고 x보다 먼저 시작	$y_{post} < x_{pre}$	4→3

- 4→3: 3과 4는 같은 부모(2)의 자식 = 형제 → cross
- **tree와 forward는 부등식이 같음** → pre/post만으로는 구분 불가 (tree 여부는 탐색 중에 기록)
- 간선 분류는 응용마다 쓸모가 다름 (cycle 검사가 대표적)

cycle과 DAG

> 방향 그래프에 cycle이 있다 $\iff$ 그래프 전체에 DFS forest를 구성하면 back edge가 발견된다 — 미방문 노드에서 다시 시작해 전체를 탐색하면 시작 순서와 관계없이 cycle이 있을 때 back edge가 발견되고, 없으면 DAG이다. 단, 단일 시작 정점에서만 탐색하면 도달할 수 없는 영역의 cycle은 놓칠 수 있다. DFS 한 번 $O(|V| + |E|)$으로 acyclic 판정.

- 슬라이드 그래프: back edge 3→1 → cycle 1→2→3→1. 3→1을 빼면 back edge 없음 → acyclic
- 간선 수로는 판단 불가 (무방향과 다름): 노드 4개·간선 6개인 cycle 있는 그래프와 DAG가 둘 다 존재
- **DAG** (directed acyclic graph) — cycle 없는 방향 그래프. 선행 과목·의존 관계 모델

DAG의 성질

> - **post 번호의 내림차순 = 선형화(linearization)** — 모든 간선은 post 번호가 더 낮은 노드로 향함
> - 유한하고 비어 있지 않은 DAG에서는 source(들어오는 간선 없음)와 sink(나가는 간선 없음)가 각각 **1개 이상** 존재한다 (여러 개일 수 있음)

- 예 (3→1 제거): post — 1:8, 2:7, 3:4, 4:6. 간선마다 post 비교: 1→2 (8>7), 1→4 (8>6), 2→3 (7>4), 2→4 (7>6), 4→3 (6>4)
- post 내림차순 8, 7, 6, 4 → **선형화 1, 2, 4, 3** — 간선 5개가 전부 왼쪽 → 오른쪽. 2→4, 4→3 때문에 순서가 고정되어 이 DAG의 선형화는 이 하나뿐

back edge ⇔ cycle (증명)

- back edge $x \to y$가 있으면 y는 x의 조상 → 트리 path $y \rightsquigarrow x$ + 간선 $x \to y$ = cycle
- 반대로 cycle이 있으면 cycle 위에서 가장 먼저 발견되는 노드 v를 잡음. cycle의 나머지 노드는 모두 v의 자손 → v로 돌아오는 마지막 간선이 back edge

source가 반드시 존재 (모순)

- source가 없다고 가정 → 모든 노드에 들어오는 간선이 있음
- 아무 노드에서 시작해 들어오는 간선을 거꾸로 계속 따라감 → 노드 수가 유한하므로 언젠가 이미 지난 노드에 도착 → cycle → DAG라는 가정과 모순 (sink도 같은 방식)

연습문제

문제 1

슬라이드 2 그래프(인접 리스트 순서는 번호 오름차순)에서 6번에서 시작한 DFS의 방문 순서, 각 노드의 pre/post, tree edge를 구하시오.

힌트 보기

① explore가 이웃을 리스트 순서대로 보고, 미방문 이웃을 만나면 즉시 그 노드로 내려감

② 스택에서 빠질 때가 post

③ 6의 유일한 이웃은 5

정답 보기

개념은 위 힌트 참고.

① 6 → 5로 내려감. 5의 리스트(2, 3, 4, 6)에서 2가 먼저

② 2의 리스트(1, 3, 4, 5)에서 1이 먼저, 1의 리스트(2, 3)에서 3 → 3 → 7 → 8까지 직선으로 내려감

③ 8, 7, 3, 1이 차례로 끝나고 되돌아온 2에서 마지막으로 4를 방문

④ pre/post: 6(1,16) 5(2,15) 2(3,14) 1(4,11) 3(5,10) 7(6,9) 8(7,8) 4(12,13)

⑤ tree edge: (6,5) (5,2) (2,1) (1,3) (3,7) (7,8) (2,4) → 7개

최종 답 **방문 순서 6, 5, 2, 1, 3, 7, 8, 4 · tree edge 7개 (= n−1)**

문제 2

같은 그래프에서 5번에서 시작한 BFS의 큐 변화와 dist를 구하고, 같은 층 간선을 모두 찾으시오.

힌트 보기

큐는 FIFO — eject한 노드의 미방문 이웃을 dist + 1로 inject. 같은 층 간선 (u, v)가 있으면 공통 조상까지 올라가 $2i + 1$짜리 홀수 cycle.

정답 보기

개념은 위 힌트 참고.

① 큐: [5] → [2 3 4 6] → [3 4 6 1] → [4 6 1 7 8] → [6 1 7 8] → [1 7 8] → [7 8] → [8] → [ ]

② 방문 순서 5, 2, 3, 4, 6, 1, 7, 8

③ dist: 5→0 / 2, 3, 4, 6→1 / 1, 7, 8→2

④ 같은 층 간선: 거리 1끼리 (2,3), (2,4) / 거리 2끼리 (7,8) → 대응 홀수 cycle: 2-3-5-2, 2-4-5-2, 3-7-8-3 (모두 길이 3)

최종 답 **dist 0: {5} · 1: {2,3,4,6} · 2: {1,7,8} · 같은 층 간선 (2,3) (2,4) (7,8)**

문제 3

방향 그래프(1→2, 1→4, 2→3, 2→4, 3→1, 4→3)를 3번에서 시작해 DFS(이웃은 번호 순)할 때 pre/post와 간선 종류, cycle 유무를 구하시오.

힌트 보기

규칙	화살표 방향으로만 따라감
간선 분류	간선 (x, y)를 검사하는 시점에 y가 아직 방문 전이면 tree · y가 스택에 있으면(탐색 중) back · y가 이미 끝났으면 forward 또는 cross(y가 x의 자손이면 forward, 아니면 cross). tree와 forward는 pre/post 구간만으로는 구분할 수 없어 탐색 중 기록이 필요하다
cycle 판정	back edge가 있으면 cycle

정답 보기

개념은 위 힌트 참고.

① 3 → 1 → 2 → 4 순으로 내려감 (3→1, 1→2, 2→4가 tree edge)

② 2→3: 3이 스택에 있음 → back. 4→3: 3이 스택에 있음 → back

③ 1→4: 4가 끝났고 1보다 늦게 시작 → forward

④ pre/post: 3(1,8) 1(2,7) 2(3,6) 4(4,5)

⑤ back edge 2개 → cycle이 있음. 예: 3→1→2→3, 3→1→4→3. 시작 노드가 달라도 back edge 존재 결론은 같음 (시작 1에서는 3→1 하나)

최종 답 **back edge 2개 (2→3, 4→3) → cycle 있음. 시작 노드와 무관하게 결론 동일**

문제 4

① (a) 연결된 무방향 그래프가 노드 9개, 간선 9개이면 cycle이 있는가

② (b) cycle 없는 무방향 그래프가 노드 9개, 간선 7개이면 연결되어 있는가

③ (c) 노드 4개, 간선 6개인 방향 그래프는 반드시 cycle이 있는가

힌트 보기

tree Theorem: connected / acyclic / $n-1$ 간선 중 둘이면 나머지 하나. 무방향은 간선 수로 cycle 판정 가능하지만 방향 그래프는 불가 — 반례 하나면 충분.

정답 보기

개념은 위 힌트 참고.

① (a) 연결 + 간선 $9 \neq n-1 = 8$ → tree가 아님 → cycle 있음

② (b) tree라면 간선이 8개여야 하는데 7개 → 연결되어 있지 않음 (연결요소 2개)

③ (c) 아님: 1→2, 1→3, 1→4, 2→3, 2→4, 3→4는 간선 6개인 DAG. 방향 그래프는 간선 수가 아니라 DFS back edge로 판단

최종 답 **(a) 있음 · (b) 연결 X · (c) 반드시는 아님 (DAG 반례 존재)**

문제 5

$n = 10^5$ 노드, $m = 3 \times 10^5$ 간선인 무방향 그래프를 인접 행렬과 인접 리스트로 저장할 때 저장 칸 수를 비교하고, 간선 (u, v) 존재 확인 비용도 비교하시오.

힌트 보기

행렬은 n^2 칸, 리스트는 $n + 2m$ 칸 (간선 두 번 저장). 확인 비용은 행렬 $O(1)$, 리스트는 짧은 쪽 차수만큼.

정답 보기

개념은 위 힌트 참고.

① 행렬: $n^2 = 10^{10}$ 칸

② 리스트: $n + 2m = 10^5 + 6 \times 10^5 = 7 \times 10^5$ 칸

③ 칸 수 비율 $10^{10} / (7 \times 10^5) \approx 14{,}000$배

④ 확인 비용: 행렬 $O(1)$, 리스트 $O(\min(\deg u, \deg v))$ (평균 차수 $2m/n = 6$)

⑤ 이 칸 수 비율은 저장 항목 수 기준이다. 실제 바이트 단위 메모리 사용량의 비율은 항목의 표현 방식과 자료구조 오버헤드에 따라 달라진다

⑥ 평균 차수 2m/n = 6은 평균일 뿐이다 — 개별 조회 비용은 두 정점의 실제 차수에 따라 달라지고, 최악의 경우 O(|V|)까지 걸릴 수 있다

최종 답 **리스트가 항목 수 기준 약 14,000배 절약 · sparse 그래프라 인접 리스트가 맞음**